

FIVE AGENT FRAMEWORKS.

A practical, evidence-led guide for explaining architecture, selecting frameworks and proving your recommendation.

LangGraph

OpenAI Agents SDK

Google ADK

Microsoft Agent Framework

CrewAI

Central teaching message: There is no universally best framework. Choose based on workflow complexity, control requirements, cloud/tooling fit, reliability and measured performance.

Includes: architecture diagrams | production use case | decision guide | benchmark | official references

The answer you should remember

All five can implement single-agent and multi-agent systems. Their biggest difference is how they structure control and collaboration.

LangGraph

Explicit state, nodes, edges, routing and checkpoints.

State-centric

OpenAI Agents SDK

Lightweight agent loop, tools, handoffs and code-driven orchestration.

Agent-centric

Google ADK

Composable agents, workflow templates and graph orchestration.

Composition-centric

Microsoft Agent Framework

Agents plus controlled enterprise workflows, orchestration and recovery.

Workflow-centric

CrewAI

Agents with roles, tasks, crews and event-driven flows.

Role / task-centric

Practical rule: Architecture description is not a performance benchmark. Vendor documentation proves a capability; controlled experiments demonstrate whether a framework is a better fit for your use case.

Five design mindsets - side by side

Use these diagrams to show that the same agentic requirement can be implemented through different primitives.

01 LangGraph

State > Node > Edge > Next Node

You model the state and explicit execution path. [LG-1, LG-2]

02 OpenAI Agents SDK

Triage Agent > Tool / Handoff > Specialist

An agent runs an LLM/tool loop and delegates with tools or handoffs. [OA-1, OA-2]

03 Google ADK

Root Agent > Sequential / Parallel / Graph

Compose specialized agents with predefined workflows or graphs. [GA-1, GA-2]

04 Microsoft Agent Framework

Agent / Executor > Workflow > Approval

Control agents and executors inside recoverable workflow graphs. [MS-1, MS-2]

05 CrewAI

Flow > Crew > Agent + Task

Separate agent teamwork from event-driven workflow control. [CR-1, CR-2]

Do not teach: "LangGraph is the only graph framework." Google ADK 2.x and Microsoft Agent Framework also support explicit graph-based workflows. The distinction is abstraction and workflow design, not exclusive graph support.

Technical comparison - core abstractions

Each column captures the framework's natural programming model, not an exclusive technical restriction.

Dimension	LangGraph	OpenAI SDK	Google ADK	Microsoft AF	CrewAI
Core building blocks	State, nodes, edges	Agent, Runner, tools, handoffs	Agents, runner, workflow, graph	Agents, executors, workflows	Agents, tasks, crews, flows
Primary style	Explicit state graph	Agent loop + Python	Hierarchy, templates, graphs	Graph + orchestration patterns	Roles + crews + flows
Who controls next step?	Graph routing / developer or model	Agent / model; Python code	Workflow engine or agents	Workflow or agent policies	Crew delegation or Flow routing
Dynamic agent collaboration	Graphs, supervisor/swarm patterns	Handoffs and agents as tools	Agent delegation / transfers	Handoff, group chat, Magentic	Hierarchical crews / delegation
Sequential / parallel	Native graph patterns	Python orchestration	Workflow components / graphs	Built-in orchestration	Crews / Flows
Explicit state focus	A core design principle	Sessions, context and code	Session state and workflow state	Workflow state / checkpoints	Flow state / persistence
Best classroom contrast	State-first	Agent-first	Composition-first	Controlled workflow-first	Role/task-first

Key insight: All five are flexible. "Natural fit" means the framework gives you the closest abstraction to the problem, not that another framework cannot implement it.

Technical comparison - production concerns

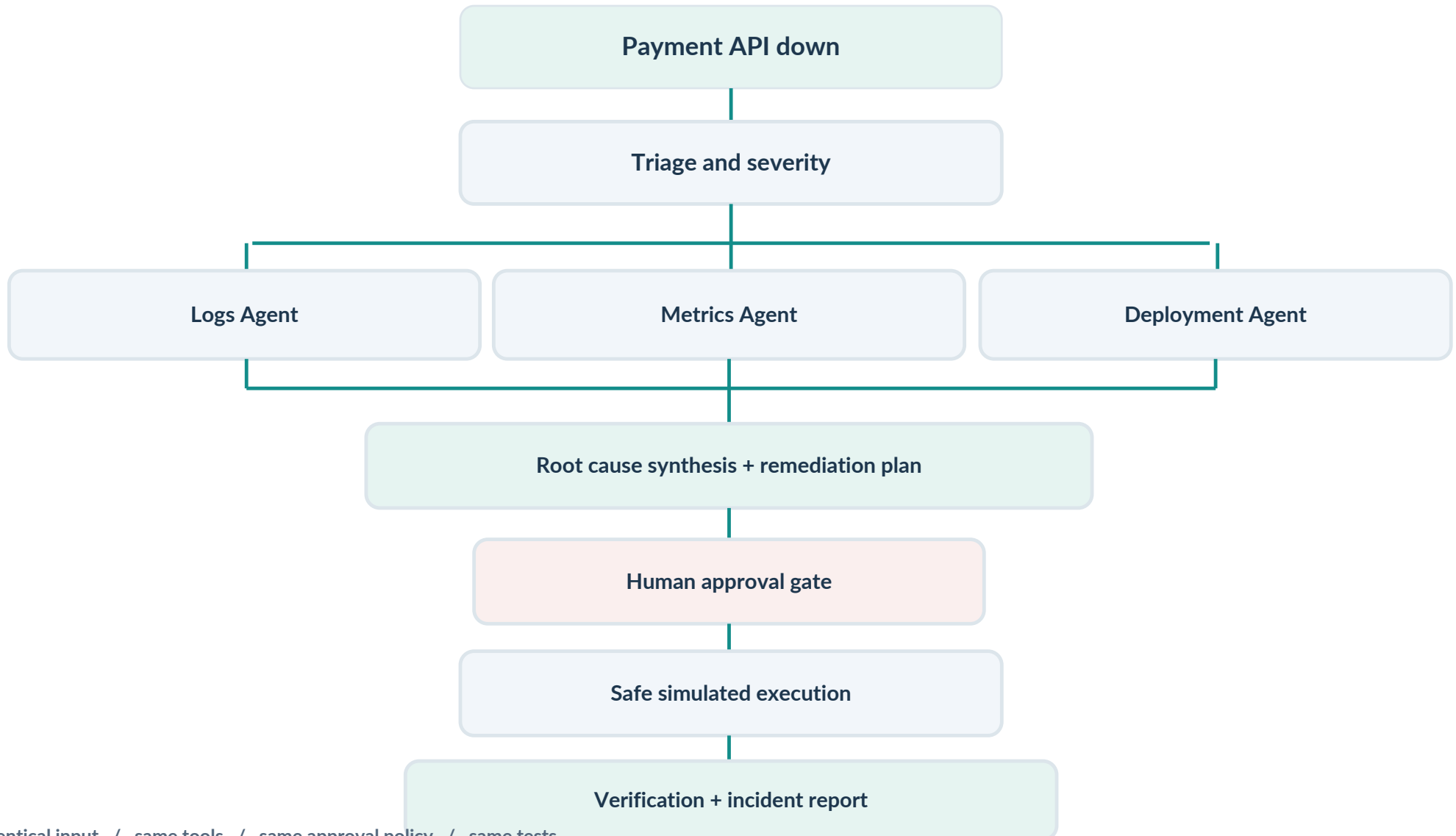
Check the current SDK version and provider integration before relying on any production-specific behavior.

Concern	LangGraph	OpenAI SDK	Google ADK	Microsoft AF	CrewAI
Memory / persistence	Checkpointers + stores	Sessions; optional durable backends	Session / memory / artifacts	Agent memory; workflow state	Memory + flow persistence
Human approval	Graph interrupts	Tool approvals / interruption	Approval and input patterns	RequestPort, tool approvals	Human feedback / Flow gates
Checkpointing	Graph checkpoints; replay	RunState + durable integrations	Workflow/session features vary by runtime	Workflow checkpoint storage	Flow/Crew persistence patterns
Tracing / debugging	LangSmith + integrations	Built-in tracing	ADK telemetry / cloud tools	OpenTelemetry-compatible	CrewAI tracing / integrations
Multi-agent ownership	Explicit state control	Manager or active specialist	Root/subagents, workflow	Executors + orchestrators	Crew roles + tasks
Cloud fit (common)	Cloud neutral / LangSmith	OpenAI-native, model adapters	Google Cloud friendly	Azure/.NET friendly	CrewAI AMP optional
Typical trade-off	More explicit workflow code	Custom complex flows use Python	Multiple workflow modes to learn	Broader enterprise surface	Task abstraction can hide detail

Accuracy note: None of the five should be called universally faster, cheaper, more secure or more reliable without controlled testing. Features and persistence semantics are not identical.

One problem. Five implementations.

Use the same tool data and safety rules in all five implementations: Payment API incident resolution.



Identical input / same tools / same approval policy / same tests

How each framework implements the case

Explain the decision-making perspective for each framework - this is the heart of the teaching comparison.

A LangGraph

Define IncidentState; graph nodes for triage, logs, metrics and deployments; parallel branches; reducer; interrupt and checkpoint.

Question: What is the current state, which node is next, and where do we resume?

B OpenAI Agents SDK

Create triage + specialist Agents. Use Agent.as_tool() or handoffs; explicitly coordinate parallel checks and approval through Python.

Question: Which agent owns the turn and who should receive the task?

C Google ADK

Compose LlmAgents with sequential/parallel workflows, or explicit ADK graph routes for conditional execution.

Question: Which reusable agent and workflow components should be composed?

D Microsoft Agent Framework

Connect agents or executors with workflow orchestration; concurrent investigation; checkpoint recovery; HITL approval.

Question: How do we make this controlled workflow observable and recoverable?

E CrewAI

Give analysts distinct Agent roles and Task objects; coordinate with a Crew; control workflow decisions in a Flow.

Question: Which agent owns which business task and how should the crew collaborate?

Which framework should I choose?

These are first frameworks to evaluate, not universal winners. Always confirm with your own prototype.

Project requirement	First option to evaluate	Why this is a reasonable starting point
Custom state, loops, retries, interruption	LangGraph	Explicit graph state, checkpoint and routing design
Lightweight assistant with tools + handoffs	OpenAI Agents SDK	Small number of agent primitives; straightforward agent loop
Gemini / Google Cloud engineering stack	Google ADK	Native Google integration plus graph / template workflows
Azure / .NET enterprise ecosystem	Microsoft Agent Framework	Multi-agent patterns, .NET support, workflow integrations
Role-based autonomous team automation	CrewAI	Roles, tasks, crews and flows match the domain
Strong workflow recovery / audit needs	LangGraph or Microsoft AF	Explicit checkpoint/recovery semantics are prominent
Complex parallel investigation	Compare all five	All can implement it; cost and runtime complexity differ
Existing OpenAI app moving to agents	OpenAI Agents SDK	Natural continuation of the existing app stack
Business process with agentic subtasks	CrewAI Flows or Microsoft AF	Mix controlled stages with autonomous execution

Decision rule: Start with project constraints: existing cloud, team language, workflow state/recovery needs, approval requirements and operational cost. Popularity alone is insufficient.

Framework selection

Ask students these questions before recommending any one SDK.

1

Already invested in Azure and .NET?

Also benchmark LangGraph for custom graphs.

Start with Microsoft Agent Framework

2

Already invested in Google Cloud / Gemini?

Also benchmark LangGraph if workflow complexity is high.

Start with Google ADK

3

Need simple tool calls, handoffs, thin SDK?

Check whether extra durable orchestration is required.

Start with OpenAI Agents SDK

4

Need deterministic, stateful custom control?

Microsoft AF is a serious alternative.

Start with LangGraph

5

Problem is naturally role- and task-based?

Compare its Flows with a lower-level orchestration.

Start with CrewAI

6

No strong requirement in any category?

Run the same benchmark instead of guessing.

Prototype 2 candidates

What should students learn first?

Teaching order is not a league table. The right depth depends on the student's target stack and job role.

01

LangGraph

Deep foundations

State, nodes, edges, routing, persistence, HITL, subgraphs and recovery.

02

OpenAI Agents SDK

Contrast a thin SDK

Implement same problem using Agent, Runner, tools, handoffs and Python orchestration.

03

Google ADK

Learn composition

Agent hierarchy, sequential/parallel templates, graph workflows and cloud integration.

04

Microsoft AF OR CrewAI

Specialize

Choose Microsoft for .NET/Azure workflows or CrewAI for task/role-centric collaboration.

Suggested final assignment: "Implement an identical incident-resolution workflow in two frameworks. Show architecture, code, failure-recovery proof and a reproducible benchmark. Justify your engineering choice."

A concise, defensible explanation when a student asks "Which framework is best?"

"There is no universally best Agentic AI framework.

All five can build single-agent and multi-agent systems. The key difference is their orchestration philosophy. **LangGraph** emphasizes explicit state and customizable graph execution. **OpenAI Agents SDK** emphasizes lightweight agents, tools and handoffs. **Google ADK** emphasizes composable agents, workflow patterns and graphs. **Microsoft Agent Framework** combines agents with enterprise workflow control and recovery. **CrewAI** focuses on role-based collaborative agent teams, tasks and flows.

We select based on project requirements, existing infrastructure, engineering skills, reliability needs and measured results - not hype or popularity.

Learn one deeply, understand the others, and defend your decision with experiments."

One-line Hinglish recap for students

"Framework best woh hai jo tumhari requirement ko simplest, safest aur most maintainable way mein solve kare - aur jiska result tum measure karke prove kar sako."

Student FAQs - quick answers

These responses distinguish documented capabilities from decisions requiring your own evidence.

Q1. Does only LangGraph support graphs?

No. Google ADK 2.x and Microsoft Agent Framework also have graph/workflow orchestration. LangGraph emphasizes state-centric low-level control.

Q2. Is OpenAI Agents SDK OpenAI-model-only?

No. The SDK supports third-party model configurations, but tools and feature compatibility vary by provider.

Q3. Are Google ADK and Microsoft AF only for their cloud?

No. They have broader execution/model integrations; Google Cloud and Azure/.NET are natural ecosystem fits, not hard exclusions.

Q4. Is CrewAI only for toy multi-agent demos?

No. Crews and Flows support more structured workflows; review persistence, control and production constraints case by case.

Q5. Is "less code" always better?

No. Measure implementation effort, visibility of control flow, debuggability, recoverability and future changes.

Q6. Which one is industry number one?

No single comparable global production market-share audit proves an unqualified winner for every company and workload.

Official documentation and evidence

Links are clickable in the PDF. Vendor documentation demonstrates technical capability; it is not an independent benchmark.

[LG-1] LangGraph overview

State-centric orchestration and runtime.

[LG-2] LangGraph persistence

Checkpointers, store, time travel, HITL.

[OA-1] OpenAI Agents SDK

Agent loop, handoffs, tracing, guardrails.

[OA-2] OpenAI multi-agent orchestration

Managers, handoffs and code orchestration.

[OA-3] OpenAI sessions

Session memory.

[GA-1] Google ADK workflow agents

Sequential, parallel and loop templates.

[GA-2] Google ADK graph routes

Graph routes and execution semantics.

[MS-1] Microsoft orchestration patterns

Sequential, concurrent, handoff, group chat, Magentic.

[MS-2] Microsoft workflow checkpoints

Checkpoint creation, storage, restore.

[MS-3] Microsoft HITL

RequestPort and tool approval.

[CR-1] CrewAI introduction

Agents, Crews and Flows.

[CR-2] CrewAI Flows

Event routing, state and persistence.